DWC · RECETARIO

JavaScript por operaciones

Para cada cosa que quieras hacer: el código. Arrays, objetos, DOM, formularios y storage.

---

0 Índice: quiero...

1. ...crear y recorrer un array
2. ...añadir y quitar elementos
3. ...buscar dentro de un array
4. ...transformar, ordenar, sumar
5. ...tocar texto (mayúsculas, cortar...)
6. ...crear y modificar objetos
7. ...hacer una clase
8. ...seleccionar y leer del DOM
9. ...crear y añadir elementos
10. ...quitar elementos y clases
11. ...escuchar eventos
12. ...leer un formulario
13. ...validar con regex
14. ...guardar datos en el navegador
15. ...recetas combinadas

1 Arrays: crear y recorrer

Crear

```
let vacio = [];
let numeros = [10, 20, 30];
let textos = ["uno", "dos", "tres"];
let mixto = [1, "dos", true, null];

let objetos = [
  { nombre: "A", precio: 10 },
  { nombre: "B", precio: 5 }
];
```

Recorrer FOR

```
// el de siempre: te da el índice
for (let i = 0; i < textos.length; i++) {
  console.log(i, textos[i]);
}

// sin índice
for (const t of textos) {
  console.log(t);
}

// con función
textos.forEach(function (t) {
  console.log(t);
});

// arrow
textos.forEach(t => console.log(t));
```

¿Cuál uso? El `for` clásico si necesitas la posición; `forEach` si solo quieres recorrer.

Longitud y acceso

```
textos.length;      // 3
textos[0];          // "uno"
textos[textos.length - 1];  // último
textos.at(-1);              // último (moderno)
```

---

2 Arrays: añadir y quitar

```
let a = ["uno", "dos", "tres"];

// AÑADIR
a.push("cuatro");          // al final
a.unshift("cero");         // al principio
a.splice(2, 0, "nuevo");   // en la posición 2, sin borrar nada
let b = a.concat(["cinco"]);   // devuelve uno nuevo

// QUITAR
a.pop();                   // quita el último
a.shift();                 // quita el primero
a.splice(2, 1);            // en la posición 2, borra 1
a.length = 0;              // vaciar todo

// quitar por condición (devuelve uno nuevo, no toca el original)
let sinDos = a.filter(x => x !== "dos");

// reemplazar uno en su sitio
a.splice(1, 1, "reemplazo");
```

Quiero	Uso	Devuelve
Añadir al final	`push(x)`	nueva longitud
Añadir al principio	`unshift(x)`	nueva longitud
Añadir en medio	`splice(pos, 0, x)`	`[]`
Quitar el último	`pop()`	el elemento
Quitar el primero	`shift()`	el elemento
Quitar en medio	`splice(pos, 1)`	array con lo borrado
Quitar por condición	`filter(x => …)`	array nuevo

---

3 Arrays: buscar

```
let a = ["uno", "dos", "tres"];

a.includes("dos");              // true / false
a.indexOf("dos");               // 1  (o -1 si no está)
a.lastIndexOf("dos");           // la última posición

// el valor que cumpla (o undefined)
a.find(x => x.startsWith("t"));
```

```
// la POSICIÓN del que cumpla (o -1)
a.findIndex(x => x.startsWith("t"));

// todos los que cumplan (array nuevo, [] si ninguno)
a.filter(x => x.length === 3);

// ¿hay alguno?  /  ¿los cumplen todos?
a.some(x => x === "dos");
a.every(x => x.length >= 3);
```

En arrays de objetos no sirve `includes("A")`: hay que buscar por propiedad con `find` o `filter`.

```
let items = [
  { nombre: "A", precio: 10 },
  { nombre: "B", precio: 5 }
];

items.find(x => x.nombre === "A");      // { nombre: "A", precio: 10 }
items.filter(x => x.precio > 6);       // [ { nombre: "A", ... } ]
items.some(x => x.precio > 9);         // true
```

---

4 Arrays: transformar, ordenar, sumar

```
// TRANSFORMAR cada elemento
a.map(x => x.toUpperCase());

items.map(x => x.nombre);                 // ["A", "B"]
items.map(x => `${x.nombre}: ${x.precio}€`);

// ORDENAR (¡muta el original!)
a.sort();                                  // alfabético
[...numeros].sort((x, y) => x - y);        // números: menor a mayor
[...numeros].sort((x, y) => y - x);        // mayor a menor
[...items].sort((x, y) => x.precio - y.precio);
[...items].sort((x, y) => x.nombre.localeCompare(y.nombre));

// SUMAR / MEDIA
let total = items.reduce((suma, x) => suma + x.precio, 0);
let media = total / items.length;

// JUNTAR Y SEPARAR
a.join(" · ");            // "uno · dos · tres"
"a,b,c".split(",");       // ["a", "b", "c"]
[...a].reverse();         // al revés (sin mutar el original)
```

> **sort muta.** Si no quieres cambiar el array original, ordena una copia: `[...array].sort(...)`.

---

5 Texto: mayúsculas, cortar, comparar

```
let t = "  Hola Mundo  ";

t.toUpperCase();          // "  HOLA MUNDO  "
```

```
t.toLowerCase();          // "  hola mundo  "
t.trim();                 // "Hola Mundo"  (quita espacios de los extremos)
t.trim().toLowerCase();   // "hola mundo"

t.includes("Mundo");      // true
t.startsWith("  Ho");     // true
t.endsWith("  ");         // true
t.indexOf("Mundo");       // 7

t.replace("Mundo", "JS"); // "  Hola JS  "  (solo la primera)
t.replaceAll("o", "0");   // todas
t.split(" ");             // trozos por el separador
t.length;                 // 14

// comparar sin importar mayúsculas
a.toLowerCase() === b.toLowerCase()
```

```
// NÚMEROS
Number("3.5");        // 3.5
Number("abc");        // NaN
parseInt("42px");     // 42
parseFloat("3.5kg");  // 3.5
isNaN(x);             // true si no es un número
(3.14159).toFixed(2); // "3.14"  (¡devuelve string!)
Math.round(3.6);      // 4
Math.max(1, 5, 3);    // 5
```

---

6 Objetos

```
// Crear
let item = { nombre: "A", precio: 10 };

// Leer
item.nombre;          // "A"
item["precio"];       // 10

// Modificar
item.precio = 12;

// Añadir una propiedad nueva
item.stock = 5;

// Borrar una propiedad
delete item.stock;

// ¿Existe la propiedad?
"precio" in item;                  // true
item.precio !== undefined;         // true

// Recorrer
for (let clave in item) {
  console.log(clave, item[clave]);
}

Object.keys(item);      // ["nombre", "precio"]
Object.values(item);    // ["A", 12]
```

```
Object.entries(item);   // [["nombre","A"], ["precio",12]]

// Copiar / mezclar
let copia = { ...item };
let nuevo = { ...item, stock: 5 };
```

7 Clases

```
class Item {
  constructor(nombre, precio) {
    this.nombre = nombre;
    this.precio = precio;
  }

  mostrar() {
    return `${this.nombre} cuesta ${this.precio}€`;
  }

  // propiedad calculada (se usa sin paréntesis)
  get caro() {
    return this.precio > 50;
  }

  // método de la clase, no del objeto
  static crear(nombre) {
    return new Item(nombre, 0);
  }
}

let a = new Item("A", 10);
a.mostrar();          // "A cuesta 10€"
a.caro;               // false
Item.crear("B");      // objeto nuevo

// Herencia
class Especial extends Item {
  constructor(nombre, precio, extra) {
    super(nombre, precio);     // llama al constructor del padre
    this.extra = extra;
  }
}
```

> **Recordatorio:** `new` es obligatorio. Sin él, `this` no es el objeto y todo falla.

8 DOM: seleccionar y leer

```
// Por id (el más rápido)
document.getElementById("caja");

// Por selector CSS: devuelve el PRIMERO
document.querySelector("#caja");
document.querySelector(".item");
document.querySelector("ul li");
```

```
// Todos los que cumplan: devuelve una lista
document.querySelectorAll(".item");      // se recorre con forEach
document.querySelectorAll("ul li");

// Buscar dentro de un elemento
let lista = document.querySelector("ul");
lista.querySelector("li");               // el primer li de dentro
lista.querySelectorAll("li");            // todos los li de dentro

// Subir al padre / bajar a los hijos
el.parentElement;
el.children;                             // hijos
el.firstElementChild;
el.nextElementSibling;
```

```
// LEER
el.textContent;            // todo el texto, tal cual
el.innerText;              // solo el visible
el.value;                  // valor de un input / select
el.href; el.src; el.id;
el.className;              // clases como string
el.classList.contains("activo");   // true / false
el.hidden;                 // true si está oculto

// ESCRIBIR
el.textContent = "Hola";   // texto plano (seguro)
el.value = "";             // vaciar un input
el.hidden = true;          // ocultar
el.style.color = "red";
```

> **No uses `innerHTML` para crear elementos.** `textContent` es texto; `innerHTML` interpreta HTML y muchos enunciados lo prohíben.

---

9 DOM: crear, añadir, mover

```
// 1. crear
let li = document.createElement("li");
let p = document.createElement("p");
let img = document.createElement("img");

// 2. rellenar
li.textContent = "Hola";
img.src = "foto.jpg";
img.alt = "Una foto";

// 3. meter en la página
let padre = document.getElementById("lista");

padre.appendChild(li);                 // al final
padre.prepend(li);                     // al principio
padre.insertBefore(li, padre.children[1]);  // antes del segundo hijo
padre.append("texto suelto");          // texto o nodo al final
```

```
// mover uno que ya existe: appendChild lo recoloca
otroPadre.appendChild(li);
```

```
// crear varios de golpe desde un array
let datos = ["uno", "dos", "tres"];
let ul = document.querySelector("ul");

datos.forEach(function (d) {
  let li = document.createElement("li");
  li.textContent = d;
  ul.appendChild(li);
});
```

---

10 DOM: quitar elementos y clases

```
// QUITAR UN ELEMENTO
el.remove();                        // se quita a sí mismo
padre.removeChild(hijo);            // desde el padre

// VACIAR UN CONTENEDOR (sin innerHTML)
while (padre.firstChild) {
  padre.removeChild(padre.firstChild);
}

// CLASES
el.classList.add("activo");
el.classList.remove("activo");
el.classList.toggle("activo");      // si está la quita, si no la pone
el.classList.contains("activo");    // true / false
el.classList.replace("viejo", "nuevo");

// quitar todas las clases
el.className = "";

// ESTILOS
el.style.color = "red";
el.style.fontWeight = "bold";
el.style.display = "none";          // ocultar
el.style.display = "block";         // mostrar
el.style.borderColor = "red";

// quitar un estilo puesto en línea
el.style.color = "";

// ATRIBUTOS
el.setAttribute("data-id", "3");
el.getAttribute("data-id");
el.removeAttribute("data-id");
el.id = "nuevo-id";
el.dataset.id = "3";                // equivale a data-id
```

11 DOM: eventos

Evento	Cuándo
`click`	Pulsar un botón, enlace, `li`...
`submit`	Enviar un formulario (botón submit o Enter)
`input`	Cada tecla que escribes
`change`	Cambia el valor y sales del campo / `select`
`mouseover` / `mouseout`	Entras / sales con el ratón (burbujean)
`mouseenter` / `mouseleave`	Igual, pero solo para ese elemento
`keydown`	Pulsas una tecla
`focus` / `blur`	Entras / sales de un campo

```
// Un botón
boton.addEventListener("click", function () {
  console.log("pulsado");
});

// Con arrow
boton.addEventListener("click", () => console.log("pulsado"));

// Un formulario: siempre preventDefault
form.addEventListener("submit", function (e) {
  e.preventDefault();
  // ...
});

// Mientras escribe
input.addEventListener("input", function () {
  console.log(input.value);
});

// Enter
document.addEventListener("keydown", function (e) {
  if (e.key === "Enter") { /* ... */ }
});

// Quitar un listener
function saludar() { console.log("hola"); }
boton.addEventListener("click", saludar);
boton.removeEventListener("click", saludar);
```

Dos formas de saber quién recibió el evento

```
// 1. listener en cada elemento: usa el propio elemento
li.addEventListener("mouseover", function () {
  li.style.fontWeight = "bold";
});

// 2. listener en el padre: usa e.target (el hijo concreto)
padre.addEventListener("mouseover", function (e) {
```

```
  if (e.target.tagName === "LI") {
    e.target.style.fontWeight = "bold";
  }
});
```

> **`padre.querySelector("li")` siempre devuelve el primero.** Si quieres el que has tocado, es `e.target`.

Detener el comportamiento por defecto

```
e.preventDefault();      // no envía el form, no sigue un enlace
e.stopPropagation();     // no sigue subiendo por los padres
```

---

12 Leer un formulario

```
<input type="text" id="nombre">
<input type="checkbox" id="acepto">
<select id="ciudad">
  <option value="">--</option>
  <option value="bi">Bilbao</option>
</select>
<input type="radio" name="sexo" value="m">
<input type="radio" name="sexo" value="f">
```

```
// Texto
let nombre = document.getElementById("nombre").value.trim();

// Checkbox
let acepto = document.getElementById("acepto").checked;   // true / false
if (!acepto) { /* error */ }

// Select
let ciudad = document.getElementById("ciudad").value;     // "" si no eligen

// Radio (puede ser null si ninguno marcado)
let sexo = document.querySelector('input[name="sexo"]:checked');
if (sexo) { console.log(sexo.value); }

// Número
let edad = Number(document.getElementById("edad").value);
if (isNaN(edad)) { /* no es número */ }

// Todos los campos de golpe
let datos = new FormData(form);
datos.get("nombre");
```

---

13 Validar con regex

Un formulario se valida escuchando `submit`, no el `click` del botón. Y siempre con `preventDefault()` para que no se envíe.

```
const form = document.getElementById("formulario");
const mensaje = document.getElementById("mensaje");
```

```
form.addEventListener("submit", function (e) {
  e.preventDefault();                      // no se envía

  let nombre = document.getElementById("nombre").value.trim();
  let email = document.getElementById("email").value.trim();

  let errores = [];

  if (!/^[a-záéíóúñü\s]{3,50}$/i.test(nombre)) {
    errores.push("Nombre: de 3 a 50 letras.");
  }
  if (!/^[\w.+-]+@[\w-]+\.[a-z]{2,}$/i.test(email)) {
    errores.push("Email no válido.");
  }

  if (errores.length > 0) {
    mensaje.textContent = errores.join(" · ");
    mensaje.style.color = "red";
  } else {
    mensaje.textContent = "Todo correcto";
    mensaje.style.color = "green";
  }
});
```

Qué significa cada símbolo

Símbolo	Qué hace	Ejemplo
`^` `$`	Empieza / termina (sin ellos, vale una parte del texto)	`/^hola$/`
`[abc]`	Uno de esos caracteres	`[aeiou]`
`[a-z]`	Un rango	`[A-Za-z]`
`[^abc]`	Ninguno de esos	`[^0-9]`
`\d`	Un dígito	`\d{4}`
`\s`	Espacio	`\s`
`+` `*` `?`	1 o más / 0 o más / opcional	`\d+`
`{3,50}`	Entre 3 y 50 veces	`[a-z]{3,50}`
`(a|b)`	Una cosa u otra	`(19|20)\d{2}`
`/i`	Ignora mayúsculas	`/^[a-z]+$/i`

Patrones listos

Qué valida	Regex
Texto de 3 a 50 letras (acentos y espacios)	`/^[a-záéíóúñü\s]{3,50}$/i`
Solo letras, sin espacios	`/^[a-záéíóúñ]+$/i`
Año de 4 cifras (1900-2099)	`/^(19|20)\d{2}$/`
Entero positivo	`/^\d+$/`

Qué valida	Regex
Email	`/^[\w.+-]+@[\w-]+\.[a-z]{2,}$/i`
Usuario 4-12 (letras y números)	`/^[a-z0-9]{4,12}$/i`
Contraseña 8+ con letra y número	`/^(?=.*[a-z])(?=.*\d).{8,}$/i`
Teléfono 9 dígitos	`/^\d{9}$/`
Código postal 5 dígitos	`/^\d{5}$/`

```
// se usa así
regex.test(valor);          // true / false

// para extraer lo que coincide
valor.match(regex);         // array o null
```

Marcar el campo en rojo

```
campo.style.borderColor = "red";   // error
campo.style.borderColor = "";      // quitar la marca

// al salir del campo (mejor que al enviar)
campo.addEventListener("blur", function () {
  this.style.borderColor = regex.test(this.value.trim()) ? "" : "red";
});
```

Si añades `required` o `pattern` al HTML, el navegador puede bloquear el envío antes de que tu JS valide. Pon `novalidate` al `<form>` si quieres validar solo con JavaScript.

---

14 Guardar en el navegador

```
localStorage.setItem("clave", "valor");   // guardar
localStorage.getItem("clave");            // leer → "valor" o null
localStorage.removeItem("clave");         // borrar una
localStorage.clear();                     // borrar todo
localStorage.length;                      // cuántas claves

sessionStorage.setItem("clave", "valor"); // igual, pero dura la pestaña
```

Solo guarda texto. Un objeto o array se convierte en `"[object Object]"` si no usas `JSON`.

```
// Guardar un objeto / array
localStorage.setItem("datos", JSON.stringify(objeto));

// Leerlo (y tener [] si no existe)
let datos = JSON.parse(localStorage.getItem("datos")) || [];

// Modificarlo
```

```
datos.push("nuevo");
localStorage.setItem("datos", JSON.stringify(datos));
```

Mostrar u ocultar según lo guardado

```
function pintar() {
  const usuario = localStorage.getItem("usuario");

  if (usuario) {
    formulario.style.display = "none";
    saludo.textContent = "Hola, " + usuario;
    botonSalir.hidden = false;
  } else {
    formulario.style.display = "block";
    saludo.textContent = "";
    botonSalir.hidden = true;
  }
}

// llamar al cargar, al guardar y al borrar
pintar();
```

---

15 Recetas combinadas

Pintar un array en una lista

```
function pintar(datos) {
  let ul = document.getElementById("lista");

  while (ul.firstChild) ul.removeChild(ul.firstChild);  // vaciar

  datos.forEach(function (d) {
    let li = document.createElement("li");
    li.textContent = d;
    ul.appendChild(li);
  });
}
```

Filtrar y volver a pintar

```
function filtrar(texto) {
  let resultado = datos.filter(function (d) {
    return d.toLowerCase().includes(texto.toLowerCase());
  });
  pintar(resultado);
}

buscador.addEventListener("input", function () {
  filtrar(buscador.value);
});
```

Añadir desde un input

```
boton.addEventListener("click", function () {
  let texto = input.value.trim();
  if (texto === "") return;              // no añadir vacío

  datos.push(texto);
  pintar(datos);
  input.value = "";                      // limpiar
});
```

Borrar el elemento que pulsas

```
lista.addEventListener("click", function (e) {
  if (e.target.tagName !== "LI") return;

  e.target.remove();
  // si quieres quitarlo también del array:
  datos = datos.filter(d => d !== e.target.textContent);
});
```

Contador que se recuerda

```
let veces = Number(localStorage.getItem("veces")) || 0;
veces++;
localStorage.setItem("veces", veces);
document.getElementById("contador").textContent = veces;
```

---

16 Chuleta de errores

Mensaje / síntoma	Qué mirar
`Cannot read properties of null`	El elemento no existe: falta `defer` o el `id` no coincide
`x is not defined`	Variable sin declarar o nombre mal escrito
`x is not a function`	El elemento es `null` o el método está mal escrito
`Unexpected token`	Falta una llave, paréntesis o comilla
`undefined` al imprimir	La función no tiene `return`
`NaN`	Estás operando con texto que no es número
`[object Object]`	Metes un objeto donde va texto
La página se recarga	Falta `e.preventDefault()` en el `submit`
Se duplican los elementos	No vacías el contenedor antes de pintar
El hover solo funciona en el primero	Usas `querySelector("li")` en vez de `e.target`

> **Regla general:** si no pasa nada, abre la consola (F12). El error te dice la línea exacta. El 90% de las veces es `null`.